RCS-AC 완성 및 PPO/SAC 공정 비교 계획서

작성일: 2026-05-03 프로젝트: RCS-AC-first 목표: CARLA 환경에서 RCS-AC 완성본이 PPO, SAC 대비 악천후, 센서 드랍, 센서 노이즈/손상 조건에서 더 높은 강건성을 보이는지 논문 수준으로 입증한다.

1. 결론 요약

현재 RCS-AC는 clean driving 안정화 후보까지는 도달했다. 특히 Round90 및 estimated scalar v6 계열은 clean 512/2048 평가에서 collision/offroad/severe early offroad/stuck 0을 달성했다. 그러나 이것은 아직 논문에서 말할 수 있는 "완성본"은 아니다.

논문용 완성본까지 남은 핵심 작업은 다음 네 가지다.

1. RCS-AC 최종 clean baseline을 Round90 estimated scalar v6 또는 그 후속 후보로 고정한다.

2. PPO와 SAC를 같은 환경, 센서, route, action wrapper, reward/termination 조건에서 clean-safe baseline으로 만든다.

3. 각 알고리즘의 clean baseline이 확보된 뒤에만 weather/corruption/sensor-dropout robustness matrix를 실행한다.

4. 절대 성능뿐 아니라 clean 대비 성능 유지율, 안전 이벤트 증가율, 센서 손상별 degradation ratio를 비교한다.

가장 중요한 해석상 주의점은 다음이다.

- RCS-AC Round90은 PPO처럼 reward만으로 처음부터 학습한 모델이 아니다.
- Round90은 expert/recovery dataset 기반 BC imitation과 recovery-specific supervised losses로 안정화된 checkpoint다.
- 따라서 PPO/SAC를 무조건 from-scratch RL로만 두면 "알고리즘 강건성"과 "사전 imitation 안정화의 이점"이 섞인다.
- 논문에서는 최소한 from-scratch RL, matched-input RL, matched-imitation+RL 계층을 분리해서 보고해야 한다.

2. RCS-AC 현재 상황

2.1 구현 상태

RCS-AC-first는 Phase 1~3 구조를 중심으로 구현되어 있다.

영역	현재 상태	논문용 판단
Multi-modal encoder	RGB, LiDAR, ego, route 입력 사용	사용 가능
Fusion gate	RGB/LiDAR/ego/route fusion 및 floor/diagnostics 구현	사용 가능
Reliability	Phase 2 reliability 모듈 구현	RCS-AC 고유 요소로 명시
Risk context	risk context 경로 구현	RCS-AC 고유 요소로 명시
Shapley contribution	Phase 3 contribution/Shapley 관련 구현	RCS-AC 고유 요소로 명시
Dropout consistency	sensor corruption/dropout consistency 목적 구현	RCS-AC 고유 요소로 명시
RGB perception actor features	RGB road/lane mask 기반 compact actor feature	RCS-AC와 matched baselines 모두에 제공 가능
Scalar actor features	lane/heading/route scalar actor input	oracle 사용 금지, estimated scalar 사용 권장
Action wrapper	target-speed longitudinal wrapper, action smoothing	모든 알고리즘 공통 조건으로 사용 가능

영역	현재 상태	논문용 판단
Runtime lateral assist	lateral recovery assist disabled in Round90	논문 비교에서 금지 또는 별도 ablation

2.2 Clean baseline 현황

현재 고정된 RCS-AC clean baseline은 Round90이다.

근거 파일:

- docs/baselines/rcs_ac_clean_baseline_round90.md
- configs/bc_frame_rgb_perception_actor_features_v2_sustain_late_recovery_round90.yaml
- checkpoints/rgb_perception_actor_features_v2_sustain_late_recovery_bc_round90/last.pt
- round90_results/eval_512_summary.json
- round90_results/eval_2048_summary.json
- round90_results/estimated_scalar_v6_carla_eval/README.md

Round90 clean 결과:

Eval	Collision	Offroad	Severe early offroad	Stuck	Max route progress	Final route progress
512-step Round90 oracle scalar	0	0	0	0	0.435897	0.435897
2048-step Round90 oracle scalar	0	0	0	0	0.410256	0.384615
512-step estimated scalar v6	0	0	0	0	clean safety preserved	clean safety preserved
2048-step estimated scalar v6	0	0	0	0	clean safety preserved	clean safety preserved

2.3 Round90이 완성본이 아닌 이유

Round90은 clean-safe baseline으로는 쓸 수 있지만, RCS-AC 최종 완성본이라고 부르기에는 아직 부족하다.

남은 이유:

- robustness matrix가 아직 실행되지 않았다.
- PPO/SAC clean-safe 비교군이 아직 준비되지 않았다.
- Round90 oracle scalar는 CARLA auxiliary lane_offset/heading_error를 actor feature로 쓰는 경로가 있어 privileged inference 논란이 있다.
- estimated scalar v6가 이 문제를 크게 줄였지만, 최종 비교 모델로 고정하려면 manifest와 robustness 결과가 필요하다.
- PPO gate는 RCS diagnostic 기준에서 BLOCK으로 남아 있다. clean safety에는 큰 문제는 아니지만, visual-center correction/risk contribution 해석에는 추가 분석이 필요하다.

3. RCS-AC 학습 방식: imitation 여부

현재 RCS-AC Round90은 학습 초기에 imitation을 사용했다고 판단해야 한다.

근거:

- Round90 checkpoint 이름이 ..._bc_round90/last.pt이다.

- Round90 manifest의 Training source가 Round90 BC로 기록되어 있다.
- Round90 config에는 bc: 블록이 있고, bc_loss_weight, steer_loss_weight, target_corrective_delta_loss_weight, visual_center_corrective_sign_loss_weight, early_severe_* loss가 포함되어 있다.
- round90_results/bc_training_log.csv는 bc_loss, steer_mse, target_corrective_delta_sign_rate, early_severe_steer_sign_rate 등을 기록한다.

따라서 Round90은 reward-only PPO/SAC와 같은 학습 출발선이 아니다. 논문에서 공정성을 유지하려면 다음 중 하나를 선택해야 한다.

1. RCS-AC의 BC/recovery imitation을 알고리즘의 구성 요소로 명시하고, PPO/SAC에도 동일한 expert dataset을 이용한 pretraining variant를 제공한다.

2. PPO/SAC는 from-scratch baseline으로 두되, 주장은 "학습 절차 전체로서의 RCS-AC pipeline이 더 robust하다"로 제한한다.

3. 두 계층을 모두 보고한다: PPO/SAC from scratch, PPO/SAC matched imitation, RCS-AC full.

논문 설득력은 3번이 가장 높다.

4. RCS-AC 완성본까지 남은 단계

Stage A: Final RCS-AC clean candidate 고정

목표:

- privileged inference 논란이 없는 RCS-AC 후보를 고정한다.
- 권장 후보는 Round90 estimated scalar v6이다.

해야 할 일:

1. Round90 oracle scalar와 Round90 estimated scalar v6를 명확히 분리한다.

2. final comparison candidate는 estimated scalar path를 기본으로 둔다.

3. 512/2048 clean eval을 같은 Windows CARLA 조건에서 재확인한다.

4. final RCS-AC manifest를 작성한다.

5. manifest에는 config, checkpoint, policy input, RCS-only modules, auxiliary target usage, runtime assist status, action wrapper status, eval metrics를 포함한다.

완료 조건:

- 512 clean: collision/offroad/severe early offroad/stuck 0
- 2048 clean: collision/offroad/severe early offroad/stuck 0
- finite in-range action
- runtime lateral assist disabled
- oracle lane_offset/heading_error가 inference input으로 들어가지 않음

Stage B: RCS-AC robustness-ready training/eval protocol 확정

목표:

- robustness 비교 전에 모든 알고리즘이 같은 external contract를 쓰도록 고정한다.

고정 조건:

- Town04
- 같은 route representation
- 같은 route length/spacing

- 같은 maneuver schedule
- clean weather baseline
- no traffic 또는 동일 traffic seed/count
- RGB/LiDAR/ego/route sensor spec 동일
- target-speed longitudinal wrapper 동일
- action smoothing 동일
- reward/termination 동일
- corruption/weather는 clean baseline 이후에만 활성화

금지:

- RCS-AC에만 유리한 route simplification
- PPO/SAC에만 불리한 sensor removal
- PPO/SAC-only reward 변경
- hidden lane-offset steering override
- runtime lateral assist를 RCS-AC 성능으로 포장

Stage C: Robustness matrix 실행

권장 robustness 조건:

그룹	조건 예시	목적
Clean	default weather, no corruption	기준 성능
Weather	rain, fog, low sun, wet road	시각/제어 강건성
RGB corruption	blackout/dropout, Gaussian noise, blur, brightness shift	카메라 손상 강건성
LiDAR corruption	point dropout, sparsity, range noise	LiDAR 손상 강건성
Combined corruption	RGB + LiDAR mild/medium	multi-modal degradation
Sensor dropout	RGB-only, LiDAR-only, random modality dropout	reliability/fusion claim 검증

측정 metric:

- collision count/rate
- offroad count/rate
- severe early offroad count
- stuck count
- max/final route progress
- route progress retention
- safety event increase
- action jitter
- recovery sign rate
- fusion weight shift under corruption
- reliability score shift under corruption
- risk context activation under near-failure
- Shapley/contribution attribution 변화

핵심 논문 지표:

```
route_retention = corrupted_max_route_progress / clean_max_route_progress
safety_retention = 1 - corrupted_safety_event_rate
robustness_gap = RCS_AC_retention - baseline_retention
```

5. PPO 공정 비교 구현 단계

5.1 PPO baseline 계층 정의

PPO는 한 종류만 두면 해석이 불공정해질 수 있다. 다음 세 계층을 권장한다.

이름	입력	초기화/학습	목적
PPO-strict	RGB/LiDAR/ego/route	from scratch PPO	순수 RL baseline
PPO-matched-input	RGB/LiDAR/ego/route + RGB perception actor features + estimated scalar features	from scratch PPO	입력 조건 공정화
PPO-matched-imitation	PPO-matched-input	같은 expert dataset으로 BC pretrain 후 PPO	RCS-AC BC 이점 분리

현재까지의 PPO matched 상황:

- PPO matched seed9 계열이 가장 근접했다.
- best matched candidate는 512 eval에서 collision 0, offroad 1, severe early offroad 0, stuck 0, max route progress 0.538462 수준까지 접근했다.
- 아직 512 hard safety gate를 통과하지 못했다.
- seed7/seed13 repeat은 seed9보다 나빴다.
- continuation/lower LR만으로는 late offroad/collision 문제가 해결되지 않았다.

5.2 PPO에서 반드시 꺼야 하는 RCS-AC 전용 요소

PPO baseline에서는 아래 기능을 inference policy에 넣지 않는다.

- reliability module
- risk context module
- Shapley/contribution alignment
- dropout consistency
- RCS-only contribution bias
- lateral recovery assist
- lane_offset runtime steering override
- privileged semantic/lane/heading input

단, 공정한 matched-input 실험에서는 다음은 허용 가능하다.

- RGB perception actor features
- estimated scalar features
- target-speed longitudinal wrapper
- action smoothing

이유: RCS-AC final candidate도 이 actor-facing representation을 사용한다면 PPO/SAC에도 같은 표현을 제공해야 입력 공정성이 맞다.

5.3 PPO 구현/안정화 순서

1. PPO-strict config를 frozen shared contract로 정리한다.
2. PPO-matched-input config를 RCS-AC estimated scalar final candidate와 같은 actor-facing input으로 정리한다.
3. 8192/16384/32768 step smoke를 여러 seed로 실행한다.
4. 512 clean eval에서 zero safety event 후보를 찾는다.
5. 512 통과 checkpoint만 2048 eval로 승격한다.
6. 실패하면 허용 튜닝만 변경한다.
7. clean-safe PPO manifest를 작성한다.

허용 튜닝:

- total timesteps
- seed
- learning rate
- entropy loss weight
- log_std_init_by_dim
- min/max std
- rollout_steps
- minibatch_size
- update_epochs
- clip_coef
- max_grad_norm

주의:

- reward scale이나 reward term을 PPO만 바꾸면 직접 비교가 깨진다.
- action wrapper를 PPO만 바꾸면 직접 비교가 깨진다.
- PPO가 clean-safe를 끝내 못 만들면, "PPO failed to obtain clean-safe baseline under shared protocol"로 보고할 수 있지만, degradation-ratio 주장은 약해진다.

5.4 PPO matched imitation variant

RCS-AC가 BC imitation을 사용했기 때문에, 논문 설득력을 위해 PPO에도 같은 dataset 기반 pretraining variant를 두는 것이 좋다.

권장 절차:

1. RCS-AC Round90에서 사용한 expert/recovery dataset을 고정한다.
2. PPO actor-only 또는 shared encoder+actor BC pretraining을 수행한다.
3. RCS-AC 전용 모듈은 여전히 꺼둔다.
4. BC 후 PPO fine-tuning을 같은 reward/환경에서 수행한다.
5. PPO matched-imitation으로 별도 표기한다.

이 variant의 의미:

- PPO가 imitation 안정화까지 받았을 때도 RCS-AC robustness가 더 높은지 확인한다.
- RCS-AC의 이점이 단순히 BC pretraining 때문인지, reliability/risk/Shapley/fusion 설계 때문인지 분리한다.

6. SAC 공정 비교 구현 단계

6.1 SAC baseline 설계 원칙

SAC도 PPO와 같은 external contract를 사용해야 한다.

공통 입력:

- RGB
- LiDAR
- ego
- route
- matched variant에서는 RGB perception actor features
- matched variant에서는 estimated scalar features

금지:

- RCS-AC reliability
- RCS-AC risk context
- Shapley
- dropout consistency
- lane_offset runtime override
- privileged simulator input
- SAC-only reward simplification

6.2 SAC 구현 구성

SAC는 off-policy actor-critic으로 구현한다.

필수 컴포넌트:

- shared observation encoder 또는 SAC 전용 encoder
- squashed Gaussian actor
- twin Q networks
- target Q networks
- entropy temperature alpha
- replay buffer
- n-step 또는 1-step TD target
- target network Polyak update
- action_dim 2
- target-speed/action smoothing wrapper는 PPO/RCS-AC와 동일

권장 config:

```
sac:
  replay_size: 100000
  warmup_steps: 1000
  batch_size: 256
  gamma: 0.99
  tau: 0.005
  actor_lr: 0.0001
  critic_lr: 0.0001
  alpha_lr: 0.0001
  target_entropy: -2.0
  update_after: 1000
  update_every: 1
  gradient_steps: 1
```

6.3 SAC 구현 순서

1. SACAgent 추가
   - actor, twin critic, target critic, alpha optimizer
2. ReplayBuffer 추가
   - obs, auxiliary obs, action, reward, done, next obs, hidden state 필요 여부 결정
3. scripts/train_sac.py 추가
   - PPO train과 같은 env factory/wrapper 사용
4. SAC baseline config 작성
   - configs/baselines/sac_clean_same_env_sensor_action_seed7.yaml
5. SAC matched-input config 작성
   - configs/baselines/sac_clean_matched_estimated_scalar_seed7.yaml
6. local smoke test
   - dummy batch update
   - finite action
   - target network update
7. Windows CARLA 2048/8192 smoke train
8. 512 clean eval
9. 2048 clean eval
10. manifest 작성

6.4 SAC matched imitation variant

PPO와 마찬가지로 SAC도 다음 두 가지를 분리한다.

- SAC from scratch
- SAC matched imitation + SAC fine-tuning

SAC matched imitation은 actor를 expert dataset으로 먼저 BC pretrain하고, critic은 SAC replay로 학습한다. 이 경우 논문 표에는 반드시 “BC-pretrained SAC”로 표시한다.

7. 공정 비교 실험 설계

7.1 최소 논문 표 구성

Algorithm	Init	Actor input	RCS-specific modules	Clean 2048 safe	Robustness matrix
RCS-AC final	BC/recovery + optional PPO	matched estimated scalar	enabled	required	required
PPO-strict	scratch	base sensors	disabled	target	optional if clean-safe
PPO-matched-input	scratch	matched	disabled	target	required if clean-safe
PPO-matched-imitation	BC + PPO	matched	disabled	target	recommended
SAC-strict	scratch	base sensors	disabled	target	optional if clean-safe

Algorithm	Init	Actor input	RCS-specific modules	Clean 2048 safe	Robustness matrix
SAC-matched-input	scratch	matched	disabled	target	required if clean-safe
SAC-matched-imitation	BC + SAC	matched	disabled	target	recommended

7.2 Clean baseline acceptance

각 알고리즘은 robustness 비교 전에 다음을 만족해야 한다.

- 512 clean eval: collision 0, offroad 0, severe early offroad 0, stuck 0
- 2048 clean eval: collision 0, offroad 0, severe early offroad 0, stuck 0
- finite in-range action
- max route progress 〉= 0.30 권장
- runtime assist 상태 문서화
- action wrapper 상태 문서화
- inference input 상태 문서화

7.3 Robustness comparison acceptance

Robustness matrix는 clean-safe checkpoint만 대상으로 한다. 각 condition은 최소 3 seeds 또는 3 route repeats를 권장한다.

비교는 다음 순서로 보고한다.

1. Clean absolute performance
2. Corrupted absolute performance
3. Clean 대비 retention
4. Safety event increase
5. Sensor corruption별 failure mode
6. RCS-AC 내부 해석 지표: reliability/fusion/Shapley 변화

7.4 논문에서 피해야 할 주장

피해야 할 주장:

- “RCS-AC가 PPO보다 무조건 우수하다”라고 말하면서 PPO는 from-scratch, RCS-AC는 BC-pretrained인 경우
- clean baseline이 약한 알고리즘과 robustness retention만 비교하는 경우
- runtime guard가 개입한 결과를 raw policy 성능으로 보고하는 경우
- oracle lane_offset/heading_error를 inference에 넣고 sensor-based robustness라고 주장하는 경우
- PPO/SAC에 matched estimated scalar를 주지 않고 RCS-AC만 scalar feature를 쓰는 경우

권장 주장:

〉 동일한 clean-safe 조건과 동일한 deployable sensor-derived actor input에서, RCS-AC의 reliability/risk/contribution/dropout-aware 구조가 PPO/SAC 대비 센서 손상 및 악천후 조건에서 더 높은 route retention과 낮은 safety event 증가율을 보였다.

8. 현재 위치 판단

현재 프로젝트 위치는 다음과 같다.

단계	상태	설명
RCS-AC 구조 구현	거의 완료	Phase 1~3 주요 모듈 구현됨
Clean driving 안정화	부분 완료	Round90/v6 clean-safe 후보 있음
Privileged scalar 제거	부분 완료	estimated scalar v6 후보 있음
PPO strict baseline	미완료	여러 near-pass 실패, clean-safe 미확보
PPO matched baseline	미완료	seed9 near-pass, offroad/collision 1 이벤트 남음
SAC baseline	시작 전	구현/훈련/평가 필요
Robustness matrix	시작 전	clean baselines 확보 후 진행
논문 결과	아직 미완성	비교군과 robustness evidence 필요

즉 현재는 RCS-AC clean baseline 후보를 확보했고, 비교군 구축으로 넘어가는 중간 지점이다. 논문 완성까지는 “RCS-AC 알고리즘이 존재한다”보다 “공정한 비교군과 robustness evidence가 존재한다”가 더 중요하다.

9. 권장 다음 실행 순서

1. RCS-AC final candidate manifest 작성
   - Round90 estimated scalar v6를 우선 후보로 고정
2. PPO matched-input clean baseline 계속 안정화
   - seed9 near-pass family 중심
   - from-scratch와 matched-imitation을 분리
3. SAC baseline 구현
   - strict와 matched-input부터
4. RCS-AC/PPO/SAC 공통 robustness eval script 정리
5. 512/2048 clean-safe checkpoint만 robustness matrix로 승격
6. weather/dropout/noise matrix 실행
7. degradation ratio 및 failure mode 분석
8. 논문 표/그림 생성

10. 최종 완료 조건

논문 작성에 들어갈 수 있는 최소 완료 조건:

- RCS-AC estimated scalar final checkpoint clean 512/2048 PASS
- PPO matched 또는 PPO matched-imitation clean 512/2048 PASS
- SAC matched 또는 SAC matched-imitation clean 512/2048 PASS
- 각 알고리즘 manifest 작성
- 최소 4개 corruption/weather condition에서 2048 eval 실행
- RCS-AC가 route retention, safety event increase, sensor dropout recovery 중 최소 두 지표에서 통계적으로 일관된 우위를 보임
- RCS-AC 내부 reliability/fusion/contribution 지표가 failure reduction과 연결되는 해석 자료 확보

이 조건을 만족하면 논문 주장은 다음 형태가 가장 안전하다.

〉 RCS-AC는 clean driving 안정화 이후, 동일한 deployable sensor-derived input과 동일한 CARLA evaluation contract에서 PPO/SAC보다 악천후 및 센서 손상 조건의 성능 저하가 작았다. 특히 reliability-aware fusion과 risk/contribution-aware regularization이 센서 드랍 상황에서 route retention과 safety preservation을 개선했다.